Infrastructure Provisioning Hardening: Terraform & Ansible Technical Change Report

Easy Hosting Platform · Boomi Atom Deployment Pipeline

Document Information

Field	Detail
Document title	Infrastructure Provisioning Hardening: Terraform & Ansible Technical Change Report
Author	Kishwanth Aderu
Reviewer / approver	Venkatesh Reddy
Date	14 October 2026
Classification	Internal
Project	Easy Hosting (multi-tenant Boomi Atom hosting platform)
Repository	`easystepin-ai/easy-hosting` on `git.meepaisa.com`
Branch	`kishwanth/devops`, created from `release/dev`
Local path	`C:\Users\kishwanth.aderu\Desktop\Easy-Hosting\easy-hosting`

Purpose

This document records every change made to the Terraform and Ansible code on the `kishwanth/devops` branch: what each file did before, what it does now, why the change was needed and how to run and verify it. It is the reference for reviewing the branch before it is committed and merged.

Scope

In scope	Out of scope
Terraform code in `terraform/aws` and `terraform/azure`	Grafana dashboards and Grafana single sign-on
Backend services that run Terraform and Ansible (`backend/apps/platform_app/services/`)	Tenant, user, role and permission management
Ansible playbook, role, config and Dockerfile in `ansible/`	Dashboard and metrics UI beyond the SSH user fix
Terminate (destroy) API, related model default and migration	Frontend pages other than the Create Environment form
SSH key, `.gitignore`, `docker-compose.yml` pin and `deploy/env.prod.example`	Production server set-up and nginx

Change summary

Area	Result
Files changed	24 modified, 8 new, 1 deleted, 4 removed from git tracking
Terraform tests	AWS 6/6 and Azure 4/4 pass with mocked providers
Blockers fixed	Shared Terraform state, empty SSH key, Azure emulator-only provider, retired Basic IP, missing NSG, two Ansible install failures
New capability	Terminate endpoint that destroys an environment's VM and Terraform workspace
Not yet done	End-to-end run on LocalStack and on a real cloud

Contents

No.	Section	What it covers	Page
1	Overview	Deployment flow and what was broken before	4
2	Part A — Terraform AWS	Files in `terraform/aws`: firewall, AZ, AMI, Elastic IP, disk, IMDSv2, variables, tests	5
3	Part B — Terraform Azure	Files in `terraform/azure`: real credentials, Standard IP, NSG, variables, lock file, tests	8
4	Part C — Backend code that runs Terraform	`terraform_service.py`, `deploy_service.py`, terminate API, model, compose pin, form	11
5	Part D — Ansible	Playbook, role tasks, defaults, `ansible.cfg`, Dockerfile, files to delete	14
6	Part E — Backend code that runs Ansible	`ansible_service.py`, token masking, Azure SSH user	17
7	Part F — Keys, git and configuration	`.gitignore`, SSH key, untracked secrets, `env.prod.example`	19
8	Verification	Checks run and what is not yet verified	20
9	Step-by-step: run and verify	Offline checks, local set-up, LocalStack, terminate, real AWS	21
10	Conclusion	What the changes achieve and the next step	23

1. Overview

These changes make Easy Hosting able to deploy a Boomi Atom VM per tenant safely: each environment now has its own Terraform state, a locked-down firewall, a working Azure path, a working Ansible install and a terminate (destroy) option.

Repository: `C:\Users\kishwanth.aderu\Desktop\Easy-Hosting\easy-hosting` · **Branch:** `kishwanth/devops` (from `release/dev`). All file paths in this document are relative to that folder.

Status: nothing is committed. Four secret files are staged for removal from git tracking (they stay on disk); every other change is unstaged so it can be reviewed in VS Code's Source Control panel.

How a deployment flows

When a super-admin clicks Deploy, the request passes through these steps in order:

1. **UI form** (`frontend/src/admin/pages/Environments/EnvironmentCreate.tsx`) sends tenant, cloud, region, size, AMI, key and Boomi details.
2. **Django view** (`backend/apps/platform_app/views/environments.py`) saves an `Environment` row and queues a Celery task.
3. **Celery task** (`backend/apps/platform_app/tasks.py`) calls `deploy_environment()` in `backend/apps/platform_app/services/deploy_service.py`.
4. **Terraform** (`backend/apps/platform_app/services/terraform_service.py` → `terraform/aws` or `terraform/azure`) creates the VM and returns its IP.
5. **Ansible** (`backend/apps/platform_app/services/ansible_service.py` → `ansible/playbooks/boomi.yml`) SSHes into the VM and starts the Boomi Atom container.
6. **Metrics** (`backend/apps/platform_app/services/metrics_service.py`) SSHes in again and calls the Boomi API to fill the dashboard.

What was broken before

Area	Problem before	Effect
Terraform state	One state file for every tenant	A second tenant's deploy reused or destroyed the first tenant's VM
AWS firewall	SSH and Atom ports open to `0.0.0.0/0`	Any internet host could attack the VMs
Azure	Provider hard-wired to the MiniBlue emulator, retired Basic IP, no NSG	Real Azure deployments were impossible
Ansible	Self-referencing variable and a broken `docker run` template	Install failed both with and without a Boomi environment ID
SSH key	`keys/demo_mcs.pem` was an empty file	Every real deployment failed at the SSH step
Teardown	No destroy path	VMs could never be removed and kept costing money
Secrets	Keys and tfvars committed to git	Credentials exposed in the repository history

2. Part A — Terraform AWS

The AWS code now builds a secure, correctly sized VM with a fixed IP in any region, and refuses bad input before anything is created.

A1. `terraform/aws/main.tf`

Defines one environment's AWS infrastructure: VPC, internet gateway, route table, subnet, security group, EC2 VM and (new) Elastic IP.

Change	Before	After	Why
Firewall	Ports 22, 7800, 9090 open to `0.0.0.0/0`	Ports 22 and 9090 open only to `var.allowed_cidrs`; 7800 removed	Bots scan every public IP for SSH; 7800 is only for Boomi Molecule clustering, which is not used
Availability zone	Hard-coded `ap-south-1a`	`data "aws_availability_zones"` picks the first AZ in the chosen region	AZ names belong to one region
AMI	Required, typed by the admin	Optional; empty = latest Ubuntu 22.04 from Canonical (`099720109477`)	AMI IDs differ per region and are easy to get wrong
AMI drift	Not handled	`lifecycle { ignore_changes = [ami] }`	"Latest Ubuntu" changes over time; a changed AMI forces a VM replace
Public IP	Automatic IP	`aws_eip` Elastic IP per VM	AWS changes the automatic IP on every stop/start
Disk	8 GB default, not encrypted	30 GB gp3, `encrypted = true`	Docker, the Atom image and its logs outgrow 8 GB
Metadata service	IMDSv1 allowed	`http_tokens = "required"` (IMDSv2)	Blocks credential theft through SSRF
Names and tags	`boomi-sg` for everyone, no tags	`${app_name}` in names; `default_tags` app, managed_by, deployment	No name clashes; cost per tenant in Cost Explorer
Version	None	`required_version = ">= 1.5.0"`	An old Terraform cannot misread the code
Duplicate output	`vm_public_ips` also in main.tf	Removed; defined once in outputs.tf	One source of truth

Real examples

- **Firewall:** VM `13.233.10.5` used to collect thousands of `Failed password for root from 45.x.x.x` lines within hours. Now only the Easy Hosting server (for example `203.0.113.10`) can open port 22; everyone else sees a closed port.
- **AZ:** choosing `us-east-1` used to fail with `Value (ap-south-1a) for parameter availabilityZone is invalid`. Now it picks `us-east-1a` automatically.

- **AMI drift:** tenant A deployed on 1 March gets `ami-0aaa`. Canonical publishes `ami-0bbb` on 15 March. Any later apply in tenant A's workspace would have destroyed and rebuilt the VM. With `ignore_changes`, the VM stays.
- **Elastic IP:** the database saved `13.233.10.5`. After a stop/start AWS handed out `3.110.44.21` and metrics failed with `Connection timed out`. The Elastic IP keeps `13.233.10.5` for life.
- **Disk:** after a few weeks of integration runs, Docker reported `no space left on device` and the Atom stopped. 30 GB gives room to grow.

A2. `terraform/aws/variables.tf`

Variable	Before	After	Why
`instance_type`	`t3.micro` (1 GB RAM)	`t3.medium` (4 GB RAM)	The Java-based Atom is killed (`OOMKilled`) or swaps on 1 GB
`ami`	Required	Optional, default `""`	Enables the Ubuntu lookup
`allowed_cidrs`	Did not exist	Required; at least one valid CIDR	Feeds the firewall; a typo fails at plan with a clear message
`root_volume_size`	Did not exist	Default 30	Disk size is configurable
`app_name`	No check	Must match `^[a-z0-9-]{3,40}$`	Clouds reject uppercase and underscores in some names
`az`	Default `ap-south-1a`	Deleted	Replaced by the AZ lookup
`ssh_key`, `environment`	Required, never used by AWS	Deleted	They made `terraform plan` fail without a tfvars file

Example: `allowed_cidrs = ["my-office"]` used to fail at apply with AWS's `CIDR block my-office is malformed`. Now `terraform plan` stops immediately: *allowed_cidrs must contain at least one valid CIDR*.

A3. `terraform/aws/outputs.tf`

Before: `vm_ips` returned the automatic public IP.
After: `vm_public_ips` returns the Elastic IP, plus `vm_instance_ids`. The backend reads `vm_public_ips`, so the saved IP never goes stale.

A4. `terraform/aws/auto.tfvars` (deleted)

Terraform only auto-loads `terraform.tfvars` and `*.auto.tfvars`, so a file named `auto.tfvars` was never read. It also held a colleague's personal SSH public key (`tharun.kumar@ESI-PC53`) in the repository.

A5. `terraform/aws/terraform.tfvars` (now local-only)

Before: tracked in git with placeholder values (`ssh_key = "your-ssh-public-key-here"`, `key_name = "demo_mcs"`).
After: removed from git tracking and gitignored. The local copy holds `key_name = "easyhosting-key"` and `allowed_cidrs = ["203.0.113.10/32"]`, a placeholder to replace with the server's public IP. Each machine needs its own values, so they must not be committed.

A6. `terraform/aws/terraform.tfvars.example` (new)

A committed template listing the values to fill in (`key_name`, `allowed_cidrs`). New team members copy it to `terraform.tfvars`.

A7. `terraform/aws/backend.tf.example` (new)

Template for keeping Terraform state in S3 in production, with `use_lockfile = true` (Terraform 1.10+). Today state is a local file on one server: if that disk dies, Terraform forgets every VM while they keep running and billing. Copy it to `backend.tf` on the production server only (gitignored) and run `terraform init -migrate-state` once. Each environment's state is then stored at `env:/env-<id>/boomi/terraform.tfstate`.

A8. `terraform/aws/tests/plan.tftest.hcl` (new)

Six automated tests with a mocked AWS provider, run by `terraform test` in seconds with no AWS account:

1. An explicit AMI is used.
2. The default size is `t3.medium` and the AZ comes from the lookup.
3. The Ubuntu lookup is used when `ami` is empty.
4. A mock apply produces an Elastic IP output.
5. An empty or invalid `allowed_cidrs` is rejected.
6. A bad `app_name` such as `Boomi_App` is rejected.

If someone later deletes the Elastic IP or opens the firewall, `terraform test` fails before anyone deploys.

3. Part B — Terraform Azure

The Azure code can now deploy to a real subscription: it uses real credentials, a Standard public IP and a network security group, while still working with the local MiniBlue emulator.

B1. `terraform/azure/main.tf`

Defines one environment's Azure infrastructure: resource group, VNet, subnet, NSG (new), public IP, NIC and VM.

Change	Before	After	Why
Credentials	Provider arguments defaulted to MiniBlue's fake values (`miniblue`, `00000000-…`, `localhost:4567`)	Arguments default to `null`, so the provider reads `ARM_CLIENT_ID`, `ARM_CLIENT_SECRET`, `ARM_TENANT_ID`, `ARM_SUBSCRIPTION_ID`	Explicit provider arguments override environment variables, so real credentials were ignored
Emulator detection	`skip_provider_registration = true` always	`is_emulator = try(length(regexall("localhost|host.docker.internal", var.metadata_host)) > 0, false)`; registration skipped only in emulator mode	Real subscriptions need resource providers registered; `try()` handles `null`
Public IP SKU	None, so Basic	`sku = "Standard"`	Microsoft retired Basic public IPs on 30 Sept 2025
Network security group	None	NSG allowing TCP 22 and 9090 only from `allowed_cidrs`, attached to the subnet; skipped in emulator mode	A Standard IP blocks all inbound traffic unless an NSG allows it
Empty IP list	Not checked	`precondition`: real Azure refuses an empty `allowed_cidrs`	Clear message instead of an Azure API error
Names	`boomi-vnet`, `boomi-pip-0`	`${app_name}-${environment}-…`, e.g. `boomi-1a2b3c4d-prod-vnet`	No clashes between tenants
Tags, disk, version	None; Standard HDD at default size	Tags on every resource; `StandardSSD_LRS` 30 GB; `required_version`	Cost tracking, faster disk, safe Terraform version

Real examples

- **Credentials:** with real service-principal values in `.env`, a deploy used to fail with `dial tcp 127.0.0.1:4567: connect: connection refused`, because it still called the emulator. Now it authenticates to the real subscription.
- **Basic IP:** creating a Basic public IP is now rejected by Azure, so every deploy stopped at the IP step. Standard succeeds.

- **NSG:** once the IP was fixed, Ansible would still get `ssh: connect to host 20.40.10.1 port 22: Connection timed out`, because the Standard IP blocked everything. The NSG lets only the Easy Hosting server in on port 22.

B2. `terraform/azure/variables.tf`

Variable	Before	After	Why
`subscription_id`, `tenant_id`, `client_id`, `client_secret`, `metadata_host`	MiniBlue defaults	`null`	Lets real `ARM_*` credentials work
`client_secret`	Plain	`sensitive = true`	Hidden in plan and apply output
`ssh_key`	Default = a container's key `root@e299715ce0fd`	Required, no default	That key did not match `keys/demo_mcs.pem`, so Ansible got `Permission denied (publickey)`
`allowed_cidrs`	Did not exist	New, CIDR format checked	Feeds the NSG
`os_disk_size_gb`	Did not exist	Default 30	Disk size
`region`, `access_key`, `secret_key`	Defined, never used	Deleted	Dead code copied from AWS
`app_name`	No check	`^[a-z0-9-]{3,40}$`	Same as AWS

B3. `terraform/azure/outputs.tf`

Content unchanged. Git shows it as modified only because of Windows CRLF versus LF line endings.

B4. `terraform/azure/.terraform.lock.hcl`

Before: `constraints = "~> 3.0"`.
After: `constraints = "~> 3.117"`, matching `main.tf`. When the code and the lock file disagree, `terraform init` rewrites the lock file. The terraform folder is mounted into the container, so every deploy would have left this tracked file modified in git.

B5. `terraform/azure/terraform.tfvars.example` (new) and local `terraform/azure/terraform.tfvars`

The example lists `ssh_key` and `allowed_cidrs`, plus commented MiniBlue settings for local emulation. The local, gitignored file was created with the new public key from `keys/demo_mcs.pem.pub` and a placeholder IP to replace.

B6. `terraform/azure/backend.tf.example` (new)

Remote-state template for Azure Storage, including the one-time `az group create`, `az storage account create` and `az storage container create` commands. Used the same way as A7.

B7. `terraform/azure/tests/plan.tftest.hcl` (new)

Four tests with a mocked Azure provider:

1. Real-Azure mode creates the VM and NSG with a Standard IP.
2. Emulator mode creates no VM and outputs `127.0.0.1`.
3. Real Azure refuses an empty `allowed_cidrs`.
4. The emulator accepts an empty list and creates no NSG.

The SSH key inside the test is a throwaway key, not a secret.

B8. `terraform/README.md` (new)

Team guide covering how the backend runs Terraform (workspaces, terminate), one-time setup, local checks with `terraform validate` and `terraform test`, LocalStack testing and moving state to S3 or Azure Storage.

4. Part C — Backend code that runs Terraform

The backend now gives every environment its own Terraform workspace, passes the admin's choices through, quotes every value safely and can destroy an environment.

C1. `backend/apps/platform_app/services/terraform_service.py`

The Python that sends Terraform commands into the `easyhosting_terraform` container.

Change	Before	After	Why
Workspace per environment	One shared state for every tenant	`terraform workspace select -or-create env-<id>`	Each environment owns only its own resources
Concurrency	Not handled	`export TF_WORKSPACE=env-<id>` for that process only	`workspace select` writes one shared file, `.terraform/environment`
Shell safety	`-var='k=v'` built by string joining	Every value passed through `shlex.quote()`; lists sent as JSON	Request data went straight into a shell command
Timeout	None	30 minutes	A hung apply blocked the only Celery worker forever
Init	`init -upgrade -reconfigure` every run	Plain `init`	`-upgrade` re-resolved provider versions on every deploy
Locking and output	Default	`-lock-timeout=5m`, `-no-color`	Waits for a busy state; no colour codes in the UI log
MiniBlue certificate	`SSL_CERT_FILE` and `/etc/hosts` rewrite on every Azure run	Only when `ARM_METADATA_HOSTNAME` contains `localhost`	On real Azure it broke TLS: `x509: certificate signed by unknown authority`
Destroy	Did not exist	`destroy()` runs `terraform destroy`, then deletes the workspace	VMs could never be removed from the app

Real examples

- **Shared state, before:** Monday, tenant ESI is deployed and Terraform records VM `i-111` (`13.233.10.5`). Tuesday, tenant Elev8 deploys with the same AMI. Terraform sees one VM that already matches, creates nothing and returns `13.233.10.5`. Ansible installs Elev8's Atom on ESI's VM and ESI's integrations stop. With a different AMI, Terraform would instead destroy ESI's VM.
- **Shared state, after:** ESI lives in workspace `env-6f1c…` with `i-111`; Elev8 lives in `env-9a2d…` with a new `i-222`. They never see each other.
- **Concurrency:** with two Celery workers, deploy A selects `env-A` at 10:00:00, deploy B selects `env-B` at 10:00:01, and A applies at 10:00:02. Without `TF_WORKSPACE`, A's VM lands in B's state. With it, A's process stays pinned to `env-A`.

- **Shell injection:** an AMI value of `ami-1'; rm -rf /workspace; echo '` used to close the quote and wipe all Terraform state. It now becomes one harmless value, `-var 'ami=ami-1; rm -rf /'`, that AWS rejects as an invalid AMI. This was tested.
- **Coloured output:** the UI log used to show `[31mError[0m`; it now shows `Error`.

C2. `backend/apps/platform_app/services/deploy_service.py` (Terraform part)

Change	Before	After	Why
Workspace	None	`workspace=workspace_for(env.id)` passed to apply and output	Turns on per-environment state
Resource names	`app_name="boomi"` for everyone	`boomi-<first 8 chars of env id>`, e.g. `boomi-1a2b3c4d`	Unique names per environment
Region and size	Ignored	`region=env.region`, `instance_type=env.instance_type`	Everything used to go to `ap-south-1` on `t3.micro` whatever the admin picked
AMI and key	Passed even when empty	Passed as `None` when empty	Falls back to the Ubuntu lookup and to `terraform.tfvars`
Azure	`environment="dev"`, no location	`environment="prod"`, `location` from the form	Production VMs were labelled dev; the chosen location was ignored
Several VMs	Silent	Warning in the log if more than one IP returns	Only the first VM is managed
Teardown	None	New `destroy_environment()`: destroy, status `terminated`, `vm_ip` cleared, log lines	Supports terminate
Team code	`set_request_status(...)` from the newest commit	Kept unchanged	No existing logic removed

C3. `backend/apps/platform_app/tasks.py`

Before: if the environment was not found, the `except` block used `env` before it existed and crashed with `NameError: name 'env' is not defined`, hiding the real problem. **After:** "not found" is handled first with a clear message, `env.save(update_fields=["status"])` saves only the status, and a new `destroy_environment_task` runs teardown in the background.

C4. `backend/apps/platform_app/views/environments.py` and `backend/apps/platform_app/urls/environments.py`

- `deploy()` now saves `region` and `instance_type` from the form onto the `Environment`.
- New `terminate_environment` view at `POST /admin/environments/<env_id>/terminate/` (super-admin only). It refuses while the environment is `provisioning` or already `terminated`.

C5. `backend/apps/platform_app/models/environments.py` and `backend/apps/platform_app/migrations/0012_alter_environment_instance_type.py` (new)

The `instance_type` default changed from `t2.micro` to `t3.medium` to match Terraform. The migration records the change, runs automatically on startup through `entrypoint.sh`, and does not touch existing rows.

C6. `docker-compose.yml`

Before: `hashicorp/terraform:latest`.
After: `hashicorp/terraform:1.10.5`. With `latest`, a new HashiCorp release could silently change the Terraform that deploys customer VMs on the next pull. Version 1.10 or later is also needed for the S3 lock file in `backend.tf.example`.

C7. `frontend/src/admin/pages/Environments/EnvironmentCreate.tsx`

Before: size defaulted to `t2.micro` and the AMI was required for AWS.
After: size defaults to `t3.medium`; the AMI is optional with the hint *Leave empty to use the latest Ubuntu 22.04*. This matches the Terraform defaults and the automatic AMI lookup.

5. Part D — Ansible

The Ansible install now works with and without a Boomi environment ID, waits for the new VM, checks that the Atom really stays up, and never prints tokens.

D1. `ansible/playbooks/boomi.yml`

The top-level playbook that installs the Atom on the new VM.

Change	Before	After	Why
Environment ID variable	`boomi_environment_id: "{{ boomi_environment_id | default('') }}"` in play vars	Removed; default `""` lives in the role defaults	A variable defined in terms of itself loops forever while Ansible resolves it
VM readiness	`gather_facts: yes`, connect immediately	`gather_facts: false`, then `wait_for_connection` (300 s), `cloud-init status --wait`, then gather facts	Terraform reports done while the VM is still booting
Environment attach login	User `boomi_api_user | default(...)` with no `BOOMI_TOKEN.` prefix; password fell back to the installer token	`BOOMI_TOKEN.<api user>` and the platform API token	Boomi rejected the old login with HTTP 401
Hidden failures	`ignore_errors: yes` on the attach	Removed; `no_log: true`	A failed attach used to look like success
Where the API call runs	On the customer VM	`delegate_to: localhost` (the Ansible container)	It is an internet API call; it does not need the VM

Real examples

- **Recursion:** deploying without an environment ID used to fail with `recursive loop detected in template string`. It now works with or without one.
- **Boot timing:** Terraform finishes at 10:00:00 and Ansible starts at 10:00:01, but SSH only starts around 10:00:30, so the old playbook failed with `UNREACHABLE! ... Connection refused`. Even with SSH up, cloud-init's first-boot upgrades held the apt lock: `Could not get lock /var/lib/dpkg/lock-frontend`. The playbook now waits for both.
- **Attach:** an admin entered environment ID Production. The deploy turned green, but in Boomi AtomSphere the atom stayed unattached, so processes deployed to Production never ran on it. It now attaches, or fails with a visible error.

D2. `ansible/roles/boomi_atom/tasks/main.yml`

The install steps run on the VM.

Change	Before	After	Why
Input validation	Checked only `boomi_atom_name`, whose default `myatom` meant it could never fail	`assert` on atom name, installer token and account ID; API user and platform token when an environment ID is set; non-Debian OS refused	Bad input stops in seconds
Process cleanup	`pkill -f "boomi"` with `ignore_errors`	Removed, with the old systemd clean-up tasks	It killed any process containing "boomi", including its own shell and a live Atom
Starting the container	Long `docker run \` shell command with Jinja `{% if %}` inside line continuations	`community.docker.docker_container` with an `env:` dictionary; `BOOMI_ENVIRONMENTID` added only when set	The old template broke when an environment ID was given
Idempotency	Every run stopped, removed and restarted the Atom	Unchanged container reported `ok` and left alone	No needless Atom restarts
Health check	`docker ps` right after start	Wait 30 s, then require running, not restarting and 0 restarts	A bad token makes the container start and then crash-loop
Atom ID lookup	One immediate query with `ignore_errors`	Up to 20 retries 15 s apart (about 5 min); only when an environment ID is set	Registration takes 1 to 3 minutes
Secrets	Tokens printed with `-v` or on failure	`no_log: true` on every task that touches a token	Tokens stay out of logs
Dependency	None	`python3-docker` added to the apt packages	The Docker module needs it on the VM

Real examples

- **Broken template:** rendered exactly as Ansible does (`trim_blocks=True`), the old command with an environment ID came out as `-e BOOMI_ENVIRONMENTID="env-123" \  -v ...`. The backslash escapes a space, so the shell passed a stray `" "` argument that Docker very likely reads as the image name, failing with `invalid reference format`. With D1's recursion bug, both paths were broken.
- **Health check:** an expired installer token used to pass, because at second 1 the container was Up. At second 8 it crashed, restarted and crashed forever. Now at second 30 Ansible sees `RestartCount: 3` and fails with *Boomi Atom container is not healthy ... Check docker logs boomi_atom on 13.233.10.5*.

D3. `ansible/roles/boomi_atom/defaults/main.yml`

Before: `boomi_atom_name: "myatom"`, which defeated validation, plus unused values from the old native install (`boomi_user`, `boomi_group`, `java_version`, `boomi_username`, `boomi_password`).

After: every input is listed and documented: required (`boomi_atom_name`, `boomi_token`, `boomi_account_id`), optional (`boomi_environment_id`, `boomi_api_user`, `boomi_platform_token`) and settings (image, container name, port, install folder, wait and retry timings).

D4. `ansible/ansible.cfg`

Before: `inventory = /ansible/inventory.ini`, `private_key_file = /ansible/keys/demo_mcs.pem` (a file that does not exist) and `remote_user = ubuntu` (wrong for Azure).
After: only general settings: `roles_path`, `host_key_checking = False`, `retry_files_enabled = False`, `timeout = 30` and `pipelining = True`. Inventory, key and user are passed per run by the backend; pipelining cuts SSH round-trips per task.

D5. `ansible/Dockerfile`

Before: `apt-get install ansible`, Ubuntu 22.04's old 2.10, plus unused `sshpass`, `unzip` and `curl`.
After: `pip3 install "ansible==9.13.0"`, a pinned release (ansible-core 2.16) that includes the `community.docker` collection D2 needs. Rebuild the image after pulling: `docker compose build ansible`.

D6. Files to delete (deletion was blocked by a permission check)

File	Why it goes
`ansible/roles/boomi_atom/templates/boomi-atom.service.j2`	systemd unit from the old native install; unused
`ansible/roles/boomi_atom/handlers/main.yml`	Restarts that old service; never triggered
`ansible/roles/boomi_atom/vars/main.yml`	Only an unused `boomi_installer_url`
`ansible/inventory.ini`	Was overwritten by every deploy, leaving git dirty; no longer used
`ansible/inventory/hosts.ini`	Empty example, never used
`docker/ansible.Dockerfile`	A second Ansible Dockerfile that compose does not use

```
git rm ansible/roles/boomi_atom/templates/boomi-atom.service.j2
ansible/roles/boomi_atom/handlers/main.yml ansible/roles/boomi_atom/vars/main.yml
ansible/inventory.ini ansible/inventory/hosts.ini docker/ansible.Dockerfile
```

6. Part E — Backend code that runs Ansible

Each Ansible run now gets its own private folder, tokens never appear on a command line or in the deployment log, and Azure metrics log in with the right user.

E1. backend/apps/platform_app/services/ansible_service.py

Prepares and runs `ansible-playbook` inside the `boomi-ansible` container.

Change	Before	After	Why
Per-run folder	One shared `/ansible/inventory.ini` and `/tmp/key.pem`	`/tmp/eh-ansible-<random>/` holding inventory, key and variables, deleted afterwards	Parallel deploys overwrote each other's inventory; the shared inventory also dirtied git
Tokens	`-e boomi_token=XXXX -e boomi_platform_token=YYYY`	Mode-600 JSON file passed as `--extra-vars @file`	Command-line arguments are visible in `ps aux` and in logs
API user	Not passed	`boomi_api_user` passed	Correct Boomi login (D1)
Config file	Relied on the working directory	`ANSIBLE_CONFIG=/ansible/ansible.cfg` set explicitly	Ansible ignores `ansible.cfg` in a world-writable folder, and a Windows bind mount looks world-writable
Timeout	None	45 minutes	A hung run blocked the worker forever
Error message	`raise Exception(result.stderr)`	Last 40 lines of stderr, or of stdout when stderr is empty	Ansible prints task failures to stdout, so the UI used to show `Deployment FAILED:` with nothing after it
Logging	`print(...)`	`logger.info(...)`	Uses Django's logging like the rest of the app

Example: two admins deploy tenants A and B at the same moment. Before, B's inventory overwrote A's file, so A's playbook could install onto B's VM. Now each run reads only its own folder.

E2. backend/apps/platform_app/services/deploy_service.py (Ansible part)

- Passes `api_user=data.get("boomi_api_user")` to `run_ansible`.
- **Token masking:** the mock path used to write the full `docker run ... -e INSTALL_TOKEN=abcd1234...` command into the deployment log, and the log stream endpoint `/logs/stream/` has no authentication. The log now shows `INSTALL_TOKEN=****`.

Example: before, anyone who knew or guessed an environment UUID could open `/admin/environments/<id>/logs/stream/` and read the customer's Boomi installer token in plain text. Now the token never reaches the log.

E3. backend/apps/platform_app/services/metrics_service.py

Before: `collect_vm_metrics(env.vm_ip)` always logged in as `ubuntu`.
After: a new `_ssh_user(env)` returns `azureuser` for Azure and `ubuntu` for AWS. Terraform creates Azure VMs

with the user `azureuser`, so every Azure metrics run used to fail with `Authentication failed` and the dashboard showed empty CPU, memory and disk figures.

7. Part F — Keys, git and configuration

Secrets and per-machine settings are now kept out of git, and a working SSH key exists for Terraform, Ansible and metrics.

F1. `.gitignore`

Before: the rule `terraform/terraform.tfvars` matched only a file sitting directly in `terraform/`, so `terraform/aws/terraform.tfvars` and every `.pem` key were committed.
After: added `**/*.tfvars` (keeping `!**/*.tfvars.example`), `**/tfplan`, `**/backend.tf`, `keys/*.pem`, `keys/*.pub`, `keys/miniblue/*.pem` and `backend/keys/`.

F2. `keys/demo_mcs.pem` and `keys/demo_mcs.pem.pub`

Before: `keys/demo_mcs.pem` was an empty file (0 bytes) tracked in git. Ansible and metrics use it to SSH into every VM, so every real deployment failed at the SSH step with `Load key "/tmp/key.pem": invalid format`.

After: removed from git tracking (staged; the file stays on disk). A new 4096-bit key pair was generated, fingerprint `SHA256:0CnhpvUs…`. Both files are gitignored. The public key must be imported into AWS as the key pair `easyhosting-key`.

F4. `deploy/env.prod.example`

Added guidance for switching to real clouds:

- **AWS:** remove `AWS_ENDPOINT_URL`, otherwise Terraform keeps talking to LocalStack.
- **Azure:** the exact `az ad sp create-for-rbac --name easyhosting-terraform --role Contributor --scopes /subscriptions/<id>` command; leave `ARM_METADATA_HOSTNAME` empty for real Azure, or set `localhost:4567` for MiniBlue.
- **Both:** a reminder to fill in the `terraform.tfvars` files.

8. Verification

The Terraform code and the Python changes pass every offline check; Ansible, the app and LocalStack have not been run yet and need a live test.

Check	Result
`terraform fmt` and `terraform validate` (AWS, Azure)	Pass
`terraform test`	AWS 6/6, Azure 4/4 pass
Offline AWS plan with the real `terraform.tfvars`	No missing or unknown variables; stops where real AWS credentials are needed, as expected
Shell commands built by `terraform_service.py` (apply, output, destroy)	Printed and syntax-checked; the injection test value is quoted safely
Changed Python files compile	Pass
Ansible YAML parses; templates render correctly	Pass: auth prefix and environment-ID logic checked
Ansible playbook actually run	Not run: Ansible cannot run on Windows and Docker Desktop was stopped
App or LocalStack deploy	Not run yet
Frontend type check	Skipped: `node_modules` is not installed

9. Step-by-step: run and verify

Run every command from the repository root. Start with the free offline checks, then LocalStack.

9.1 Offline checks (no Docker, no cloud, free)

```
cd terraform/aws && terraform init -backend=false && terraform validate && terraform test

cd terraform/azure && terraform init -backend=false && terraform validate && terraform test
```

Expected: `Success! The configuration is valid.`, then 6 passed (AWS) and 4 passed (Azure).

9.2 One-time local setup

1. Start Docker Desktop and wait for "Engine running".
2. Replace `203.0.113.10/32` in `terraform/aws/terraform.tfvars` and `terraform/azure/terraform.tfvars` with your public IP. Any valid IP works for LocalStack.
3. Create `docker-compose.override.yml` in the repository root (gitignored) with a `postgres:15-alpine` service named `easyhosting_postgres` (user, password and database `easyhosting`), and add `postgres` to the backend's `depends_on`.
4. Create `backend/.env` with `DB_HOST=postgres`, `DB_PORT=5432`, the `easyhosting` database values, `BASE_DOMAIN=localhost`, `CORS_ALLOWED_ORIGINS=http://localhost:3000`, `MOCK_ANSIBLE=True` and the LocalStack AWS values (`AWS_ACCESS_KEY_ID=test`, `AWS_SECRET_ACCESS_KEY=test`, `AWS_DEFAULT_REGION=ap-south-1`, `AWS_ENDPOINT_URL=http://localstack:4566`).
5. Rebuild the Ansible image and start the stack: `docker compose build ansible`, then `docker compose up -d --build`.
6. Create the super-admin: `docker compose exec backend python manage.py createsuperuser`.

9.3 Prepare LocalStack

1. Copy the public key: `docker cp keys/demo_mcs.pem.pub easyhosting_localstack:/tmp/k.pub`
2. Import the key pair: `docker exec easyhosting_localstack awslocal ec2 import-key-pair --region ap-south-1 --key-name easyhosting-key --public-key-material fileb:///tmp/k.pub`
3. Get an AMI ID LocalStack accepts: `docker exec easyhosting_localstack awslocal ec2 describe-images --region ap-south-1 --query "Images[0].ImageId" --output text`

9.4 Verify with the command line

Check	Command	Pass when
One workspace per environment	`docker exec easyhosting_terraform sh -c "cd /workspace/aws && terraform workspace list"`	One `env-<id>` per environment
Resources in one environment	`terraform state list` with `TF_WORKSPACE=env-<id>` and the LocalStack variables	8 resources, including `aws_eip.vm[0]`
IP matches the UI	`terraform output` with `TF_WORKSPACE=env-<id>`	`vm_public_ips` equals the IP in the UI

Check	Command	Pass when
VM settings	`terraform state show 'aws_instance.vm[0]'`	`t3.medium`, `http_tokens = "required"`, 30 GB, `encrypted = true`
Firewall	`terraform state show aws_security_group.sg`	Only your IP in `cidr_blocks`; no port 7800
VMs in LocalStack	`docker exec easyhosting_localstack awslocal ec2 describe-instances --region ap-south-1`	One VM per environment, named `boomi-<id>-vm-0`

9.5 Verify terminate

1. Copy your access token from the browser (F12 → Application → Local Storage → `token`).
2. Run `curl -X POST -H "Authorization: Bearer <token>" http://localhost:5000/admin/environments/<env_id>/terminate/`.
3. Confirm the status becomes terminated, the log shows `Environment destroyed`, its workspace is gone from the list, and the other environment is untouched.

9.6 Real AWS (costs money)

1. In `backend/.env` set real AWS keys, delete `AWS_ENDPOINT_URL` and set `MOCK_ANSIBLE=False`.
2. Import `keys/demo_mcs.pem.pub` in the AWS console (Mumbai, ap-south-1) as key pair `easyhosting-key`.
3. Set `allowed_cidrs` to the public IP of the machine running the backend, because Ansible connects from there.
4. Deploy from the UI, then check EC2 → Instances, the Security, Storage and Details tabs, and EC2 → Elastic IPs.
5. Terminate the environment when finished so the VM stops costing money.

10. Conclusion

With these changes, Easy Hosting can deploy each tenant's Boomi Atom VM in isolation: no deployment can reuse, overwrite or destroy another tenant's infrastructure.

The work fixed four kinds of problem:

- **Correctness:** one Terraform workspace per environment, workspace pinning for parallel runs, a fixed Elastic IP, AMI drift protection, and an Ansible role that works with and without a Boomi environment ID.
- **Security:** firewall and NSG limited to known IPs, IMDSv2, encrypted disks, safe quoting of every value, tokens kept out of command lines and logs, and keys and tfvars removed from git.
- **Cloud coverage:** real Azure now works through `ARM_*` credentials, a Standard public IP and an NSG, while LocalStack and MiniBlue still work for free local testing.
- **Operations:** a terminate endpoint that destroys VMs and their workspaces, timeouts on Terraform and Ansible, clear error messages, pinned tool versions, and automated Terraform tests.

The Terraform code passes all offline checks (AWS 6/6, Azure 4/4 tests). The changes have not yet run end to end, so the next step is the LocalStack test in section 9. A real cloud deployment should follow only once that test passes.